Ron Hensley

ronkhensley@gmail.com

Practice

Challenge: Code-Level Practice with Functions, Loops, Lists, and Dictionaries in Python

Fri, Jun 19
Welcome, Ron! Glad to see you jumping into a challenge session.

Today's all about putting your Python skills to the test — applying what you already know in a fun way.

Are you looking to practice challenge questions from your current Python course, or do you want to try something from a different course?

Yesterday
ice finshed py 101 and py 104 and start py 102 i 4 days randon is fine

Nice, Ron — you've been on a roll 👊

You finished PY101 and PY104 and you're starting PY102 in 4 days, and you're OK with random.

Since you said "random is fine", let me just check one thing:

do you want this challenge to focus on PY101 (intro Python), or do you also want to mix in PY104 topics?

104 is more challenging to e so it for sure

Got it — we'll focus on PY104 then, and keep it challenging 👍

Since I can't pull the detailed syllabus right now, let's base this on the big themes you've seen:

functions & main()
loops with helper functions
lists & dictionaries (including list of dicts / dict of lists)
planning before coding: specs, pseudocode, incremental implementation
For today's random challenge, which sounds better:

focus more on planning & pseudocode,
or

focus more on code-level stuff with functions, loops, and dictionaries?
code level

Nice, code-level it is 👌

Here's what I suggest we target in PY104 for this challenge:

writing and using functions (including main() and helper functions)
using loops that call functions
working with lists and dictionaries, including passing them into functions
Does that list match what you want to practice, or do you want to add/remove anything before I lock it in?

perfect

Awesome, your goals are set: code-level practice with functions, loops that call functions, and lists/dicts passed into functions ☑

I’m spinning up a personalized challenge session around that now — it should be ready in under a minute.

Lean into the struggle a bit; that’s where the real growth happens.

Nice. Then here’s the game plan for this session:

we’ll work on functions that call each other,
then loops that use helpers,
then lists/dicts passed into functions,
and we’ll finish with a small pipeline-style mini project and a quick reflection.

You’re the center here: if you want to go slower, skip something, or crank difficulty up, just say it and I’ll adapt on the fly.

Also, small mistakes (indexes, off□by□one, weird outputs) are totally normal — if anything looks off, call it out and we’ll fix it together.

All good with this flexible setup?

Let’s start round 1 – warm-up: predict the output.

Here’s the code:

Python

```
def add_exclamation(text):
    return text + "!"

def shout(text):
    return add_exclamation(text.upper())

def main():
    result = shout("hello")
    print(result)

main()
```

Without running this code anywhere, what exact text gets printed to the screen?

HELLO!

Nice, that’s exactly right: HELLO! ☑

You chained the functions correctly.

Round 2 – return or print?

Here’s the starter code:

Python

```
def double_and_print(x):
    result = x * 2
    print(result)

def main():
    double_and_print(5)

main()
```

I want double_and_print to return the value instead of printing it, so main can decide what to do.

Please paste only the corrected function body for double_and_print (just the lines inside it, not the def line).

o course the cal needs print(f'{double_and_print(5)}') o whatevr to deal with result

result = x * 2
return(result)

Yep, exactly — once it returns, the caller (like main) would do the print(...) part.

For this round, the function body you wrote should look like this (fixing just spacing/indent):

Python

```
    result = x * 2
    return result
```

Round 3 - helper inside a loop.

Here’s the starter code:

Python

```
numbers = [1, 2, 3, 4]

def square(n):
    # TODO: finish this helper
    pass

def main():
    squared = []
    for num in numbers:
        # TODO: use square() here and append to squared
        pass

    print(squared)

main()
```

Your job:

Finish the square function.
Finish the for loop body so squared becomes [1, 4, 9, 16].
Please paste the full updated code for square and main (you can omit the numbers line).

```
numbers = [1, 2, 3, 4]

def square(n):
value = n * n
return value
# TODO: finish this helper

def main():
squared = []
for num in numbers:
squared.append(square(num))
# TODO: use square() here and append to squared

print(squared)

main()
```

Love how you wired square into the loop — that’s exactly the pattern I want you to get comfortable with ☑

Round 4 - debug: scope slip (NameError).

Here’s a broken example:

Python

```
def apply_discount(price):
```

```
    final_price = price - (price * discount)
    return final_price

def main():
    discount = 0.1
    original = 50
    print(apply_discount(original))

main()
```
This crashes with a NameError about discount.

Your job: fix it by changing the function signature and its use (no globals).

Reply with the patched version of apply_discount and main only.

Python

```
def apply_discount(price,loc_discount):
    final_price = price - (price * loc_discount)
    return final_price

def main():
    discount = 0.1
    original = 50
    print(apply_discount(original,discount))

main()
```
Nice fix. You pulled discount into the parameter list and passed it from main — clean and explicit ☑

Round 5 – design: list of dicts builder.

Write a function build_records(names, scores) that returns a list of dictionaries like:

Python

```
[
    {"name": "Alice", "score": 95},
    {"name": "Bob", "score": 88}
]
```
Assume names and scores are same length lists.

Limit yourself to about 6 lines inside the function, no globals.

Please open the editor, write just this function:

Python

```
def build_records(names, scores):
    # your code here
```
Then use Run & Submit with a small test if you want (e.g. print the result), and send me the code.

Python

```
name_list = ["Bob", "Jane", "Peter", "Janet"]
scores_list = [20, 40 , 60, 80]
def build_records(names, scores):
    list_of_dicts = []
    for i in range(len(names)):
        named = names[i]
        scored = scores[i]
        dict = { named: scored }
        list_of_dicts.append(dict)
    return list_of_dicts
print(f'{build_records(name_list,scores_list)}')
```
Nice, Ron — you built the list of dicts correctly and wired the loop well ☑

Only tiny thing: the spec I gave was "name": ..., "score": ... style, but your version

is still a solid pattern, so I’m counting this as a win for this round.

Quick follow-up of the same type (just to lock it in):

Can you write a new version of build_records that matches this exact structure:

Python

```
[
    {"name": "Bob", "score": 20},
    {"name": "Jane", "score": 40},
    ...
]
```

So:

keys must be "name" and "score"
values come from the two lists
Please use the editor again, write:

Python

```
def build_records(names, scores):
    # your code
```

Add a quick test at the bottom if you like, hit Run to check, then Run & Submit, and send me the code.

Python

```
name_list = ["Bob", "Jane", "Peter", "Janet"]
scores_list = [20, 40 , 60, 80]
def build_records(names, scores):
    list_of_dicts = []
    for i in range(len(names)):
        named = names[i]
        scored = scores[i]
        dict = {"name": named, "score": scored }
        list_of_dicts.append(dict)
    return list_of_dicts
print(f'{build_records(name_list,scores_list)}')
```

That’s perfect. You hit the exact structure and your loop is clean ☑

Round 6 – refactor: add main().

Here’s a top-level script:

Python

```
names = ["Bob", "Jane", "Peter", "Janet"]
scores = [20, 40, 60, 80]

def build_records(names, scores):
    list_of_dicts = []
    for i in range(len(names)):
        named = names[i]
        scored = scores[i]
        d = {"name": named, "score": scored}
        list_of_dicts.append(d)
    return list_of_dicts

records = build_records(names, scores)
print(records)
```

Refactor it so:

all the top-level “work” happens inside main()
and you add the usual:

Python

```
if __name__ == "__main__":
```

```
    main()
```

Please open the editor, refactor this whole script, then Run & Submit and send me the new version.

not sure if this is what uou meant by top level but

Yeah, that’s exactly what I meant by top-level: code that runs immediately when the file is imported/run (like your records = ... and print(records)).

Let’s do the refactor now.

Take this script:

Python

```
names = ["Bob", "Jane", "Peter", "Janet"]
scores = [20, 40, 60, 80]

def build_records(names, scores):
    list_of_dicts = []
    for i in range(len(names)):
        named = names[i]
        scored = scores[i]
        d = {"name": named, "score": scored}
        list_of_dicts.append(d)
    return list_of_dicts

records = build_records(names, scores)
print(records)
```

Refactor it so it becomes something like:

define build_records
define main() that:
sets up names and scores
calls build_records
prints the result
add the if __name__ == "__main__": main() guard at the bottom.
Please open the editor, write the refactored version, hit Run to check output, then Run & Submit and send me the code.

Python

```
names = ["Bob", "Jane", "Peter", "Janet"]
scores = [20, 40, 60, 80]

def build_records(names, scores):
    list_of_dicts = []
    for i in range(len(names)):
        named = names[i]
        scored = scores[i]
        d = {"name": named, "score": scored}
        list_of_dicts.append(d)
    return list_of_dicts
def main():
    records = build_records(names, scores)
    print(records)

if __name__ == "__main__":
    main()
```

Nice, that refactor is spot on — behavior stayed the same, but now it’s organized under main() ☑

Round 7 – transformation: dict of lists grouper.

Write a function group_by_initial(words) that:

takes a list of strings like ["apple", "art", "banana", "boat"]
returns a dict like:

```
{
    "a": ["apple", "art"],
    "b": ["banana", "boat"]
}
```

Rules:

key = first letter of the word (lowercase is fine, assume input is lowercase)
value = list of all words starting with that letter.
Please open the editor, write:

Python

```
def group_by_initial(words):
    # your code
```

Add a quick test at the bottom (call group_by_initial and print the result), hit Run, then Run & Submit and send me the code.

Python

```
str_list = ["apple", "art", "banana", "boat"]
def group_by_initials(words):
    loc_dict = {}
    for word in words:
        loc_dict.setdefault(word[0],[]).append(word)
    return loc_dict
def main():
    print(f'{group_by_initials(str_list)}')

if __name__ == "__main__":
    main()
```

That’s really good, Ron. Using setdefault is a nice touch ☑

Only mismatch is the name: I said group_by_initial, you wrote group_by_initials, but the idea is 100% there.

Quick check-in: how’s the pace so far?

Do you want to keep this same difficulty for the mini□project, or push it a bit harder?

Next up is round 8 – mini-project: pipeline all together.

Here’s your sample data:

Python

```
purchases = [
    {"customer": "Alice", "amount": 25.0},
    {"customer": "Bob", "amount": 10.0},
    {"customer": "Alice", "amount": 40.0},
    {"customer": "Bob", "amount": 5.0},
    {"customer": "Charlie", "amount": 12.5},
]
```

Goal: write a program that prints:

Text

```
Alice -> 65.0
Bob -> 15.0
Charlie -> 12.5
```

in descending order by total amount.

Requirements:

Use at least 3 functions (single-responsibility style), plus main().
No global variables for the data processing (you can define purchases inside main).
main() should orchestrate the pipeline.

Suggested structure (you can change names if you want):

compute_totals(purchases) → returns {"Alice": 65.0, ...}
sort_totals(totals_dict) → returns a sorted list of (customer, total) pairs
print_summary(sorted_totals) → prints the lines
main() → sets up data, calls these in order.
Please open the editor, code this program, use Run to check, then Run & Submit and send me the full code.

it seems fine

Nice, we'll keep this difficulty level then 👍

Here's the mini‑project again in short form:

You have:
Python

```
purchases = [
    {"customer": "Alice", "amount": 25.0},
    {"customer": "Bob", "amount": 10.0},
    {"customer": "Alice", "amount": 40.0},
    {"customer": "Bob", "amount": 5.0},
    {"customer": "Charlie", "amount": 12.5},
]
```

Goal: print
Text

```
Alice -> 65.0
Bob -> 15.0
Charlie -> 12.5
```

sorted by total (largest first).

Use 3+ functions plus main(), for example:
compute_totals(purchases) → dict
sort_totals(totals_dict) → sorted list of pairs
print_summary(sorted_totals) → does the printing
main() → wires it all together
Now jump into the editor, write the full program, hit Run to make sure it works, then Run & Submit and send me the code.

Today
Python

```
import copy
purchases = [
    {"customer": "Charlie", "amount": 12.5},
    {"customer": "Alice", "amount": 25.0},
    {"customer": "Bob", "amount": 10.0},
    {"customer": "Alice", "amount": 40.0},
    {"customer": "Bob", "amount": 5.0}
]


def compute_totals(loc_list):
    """ Takes list of dictionaries of customer/amount pairs
        Createes a list of dicts comtaining 1 entry per customer
        paired with the total of all amounts for that customer
    """
    loc_dict_list = []  # New list of dictionaries for results
    tmp_dict =  {}      # Temp scratch dictionary

    # Loop though each dictioary in the list
    for lists in loc_list:
        found = False
        # Create a dictionary entry for "customer"/"amount"
        tmp_dict["customer"] = lists["customer"]
        tmp_dict["total"] = lists["amount"]
```

```
        # Check if this customer is in new list of dicts
        for key in loc_dict_list:
            if lists["customer"] == key["customer"]:
                found = True
        # If not found add a new entry for customer
        # using deep copy as were goig to reuse tmp_dict
        if not found:
            loc_dict_list.append(copy.deepcopy(tmp_dict))
        # Else was found so locate the dict in new list
        # And add the amount to the running total
        else:
            for ldicts in loc_dict_list:
                if ldicts["customer"] == lists["customer"]:
                    ldicts["total"] += lists["amount"]
    return loc_dict_list

def sort_totals(loc_dict):
    """ Given a list of dictionaries, sorts the list
        entries by the dicionary key ["total"] high to low
    """
    loc_dict.sort(key=lambda x: x["total"], reverse=True)
    return loc_dict

def print_summary(loc_dict):
    """ Formats and prints the summary results """
    for ldict in loc_dict:
        print(f'{ldict["customer"]} -> {ldict["total"]}')
    return

def main():
    my_dict = compute_totals(purchases)
    sorted_totals = sort_totals(my_dict)
    print_summary(sorted_totals)


if __name__ == "__main__":
    main()
```

Nice work, Ron. That’s a legit little pipeline you built there ☑

You used multiple helpers, kept main() as the conductor, and got the exact output.

Round 9 – quick reflection.

In 2 short sentences:

Where did your code (especially in compute_totals or the pipeline) feel a bit clunky or longer than you’d like?
One idea you have to simplify or clean it up next time?
locating ditionary keys inside a list of dicts im sure thers a shorter ore diret method

Yeah, that makes sense — that inner looping over the list of dicts to find a customer is the “clunky” part.

A simpler approach next time would be to first build a plain dict like {"Alice": 65.0, ...} and only convert to a list of dicts at the end.

Last one, round 10 – edge□case thinking.

Name two edge cases where your mini□project might behave weirdly or break, and say briefly how you’d guard against each one.

Just a short list is fine.

it currently does not check that the amounts are umers or that there is a list at all

Those are solid edge cases to spot:

if amount isn’t a number → code could crash or add strings; you could validate isinstance(lists["amount"], (int, float)) and maybe skip or raise an error.
if purchases isn’t a list (or is None) → the for loop fails; you could check if not purchases: return [] or verify isinstance(purchases, list) at the start.
Let me give you a quick wrap-up:

Strengths I saw:
very comfortable with writing and calling functions, including main()
good use of loops + helpers (square, compute_totals, etc.)
strong handling of lists and dicts, and you even used setdefault and a custom sort key
you naturally think about edge cases, which is more advanced than just “making it work”.
Two improvement targets:
Simplify data aggregation: instead of “list of dicts + inner search”, lean on a plain dict keyed by customer, then transform at the end.
Keep function names consistent with specs (group_by_initial vs group_by_initials) so code matches requirements exactly.
Next good step for you:

re□implement compute_totals using a dict like totals = {} and do:
Python

```
  for p in purchases:
      name = p["customer"]
      amount = p["amount"]
      totals[name] = totals.get(name, 0) + amount
```

Then, if you want, we can do another session just focused on refactoring your existing code to cleaner patterns.

Before we close, are you satisfied with today’s practice and good to stop here, or is there one more small thing you want to drill?

Python

```
import copy
purchases = [
    {"customer": "Charlie", "amount": 12.5},
    {"customer": "Alice", "amount": 25.0},
    {"customer": "Bob", "amount": 10.0},
    {"customer": "Alice", "amount": 40.0},
    {"customer": "Bob", "amount": 5.0}
]


def compute_totals(loc_list):
    """ Takes list of dictionaries of customer/amount pairs
        Createes a list of dicts comtaining 1 entry per customer
        paired with the total of all amounts for that customer
    """
    loc_dict_list = []  # New list of dictionaries for results
    tmp_dict =  {}      # Temp scratch dictionary

    # Loop though each dictioary in the list
    assert (len(loc_list) >= 1), f'Error: Length of loc_list was empty'
    for lists in loc_list:
        found = False
        chk_type = type(lists["amount"])
        assert (chk_type in (float,int)), f'Error: amount {lists["amount"]} is not a num
        # Create a dictionary entry for "customer"/"amount"
        chk_type = type(lists["amount"])
        assert (chk_type in (float,int))
        tmp_dict["customer"] = lists["customer"]
        tmp_dict["total"] = lists["amount"]

        # Check if this customer is in new list of dicts
        for key in loc_dict_list:
            if lists["customer"] == key["customer"]:
                found = True
```

```
        # If not found add a new entry for customer
        # using deep copy as were goig to reuse tmp_dict
        if not found:
            loc_dict_list.append(copy.deepcopy(tmp_dict))
        # Else was found so locate the dict in new list
        # And add the amount to the running total
        else:
            for ldicts in loc_dict_list:
                if ldicts["customer"] == lists["customer"]:
                    ldicts["total"] += lists["amount"]
    return loc_dict_list

def sort_totals(loc_dict):
    """ Given a list of dictionaries, sorts the list
        entries by the dicionary key ["total"] high to low
    """
    loc_dict.sort(key=lambda x: x["total"], reverse=True)
    return loc_dict

def print_summary(loc_dict):
    """ Formats and prints the summary results """
    for ldict in loc_dict:
        print(f'{ldict["customer"]} -> {ldict["total"]}')
    return

def main():
    my_dict = compute_totals(purchases)
    sorted_totals = sort_totals(my_dict)
    print_summary(sorted_totals)


if __name__ == "__main__":
    main()
```

Nice, you went straight into adding validation, I like that instinct.

The error you hit is from this line:

Python

```
assert (chk_type in (float,int)), f'Error: amount {lists["amount"]} is not a num
```

The f-string is missing the closing quote at the end of the line, so Python thinks the string never ends.

You can fix and simplify those checks like this:

Python

```
    assert len(loc_list) >= 1, "Error: Length of loc_list was empty"
    for lists in loc_list:
        found = False
        chk_type = type(lists["amount"])
        assert chk_type in (float, int), f'Error: amount {lists["amount"]} is not a
        number'
        # Create a dictionary entry for "customer"/"amount"
        tmp_dict["customer"] = lists["customer"]
        tmp_dict["total"] = lists["amount"]
```

Note:

closed the f-string with ' at the end
removed the duplicated chk_type / assert block.
Can you update your code in the editor with this fix (or similar), hit Run to confirm it works, then Run & Submit again so I can see the clean version?

Python

```
import copy
purchases = [
```

```
    {"customer": "Charlie", "amount": 12.5},
    {"customer": "Alice", "amount": 25.0},
    {"customer": "Bob", "amount": 10.0},
    {"customer": "Alice", "amount": 40.0},
    {"customer": "Bob", "amount": 5.0}
]


def compute_totals(loc_list):
    """ Takes list of dictionaries of customer/amount pairs
        Createes a list of dicts comtaining 1 entry per customer
        paired with the total of all amounts for that customer
    """
    loc_dict_list = []  # New list of dictionaries for results
    tmp_dict =  {}      # Temp scratch dictionary

    # Loop though each dictioary in the list
    assert (len(loc_list) >= 1), f'Error: Length of loc_list was empty'
    for lists in loc_list:
        found = False
        chk_type = type(lists["amount"])
        assert (chk_type in (float,int)), f'Error: amount {lists["amount"]} is not a
        number'
        # Create a dictionary entry for "customer"/"amount"
        chk_type = type(lists["amount"])
        assert (chk_type in (float,int))
        tmp_dict["customer"] = lists["customer"]
        tmp_dict["total"] = lists["amount"]

        # Check if this customer is in new list of dicts
        for key in loc_dict_list:
            if lists["customer"] == key["customer"]:
                found = True
        # If not found add a new entry for customer
        # using deep copy as were goig to reuse tmp_dict
        if not found:
            loc_dict_list.append(copy.deepcopy(tmp_dict))
        # Else was found so locate the dict in new list
        # And add the amount to the running total
        else:
            for ldicts in loc_dict_list:
                if ldicts["customer"] == lists["customer"]:
                    ldicts["total"] += lists["amount"]
    return loc_dict_list

def sort_totals(loc_dict):
    """ Given a list of dictionaries, sorts the list
        entries by the dicionary key ["total"] high to low
    """
    loc_dict.sort(key=lambda x: x["total"], reverse=True)
    return loc_dict

def print_summary(loc_dict):
    """ Formats and prints the summary results """
    for ldict in loc_dict:
        print(f'{ldict["customer"]} -> {ldict["total"]}')
    return

def main():
    my_dict = compute_totals(purchases)
    sorted_totals = sort_totals(my_dict)
    print_summary(sorted_totals)


if __name__ == "__main__":
    main()

Nice, Ron — tests pass and you've now got basic validation wired in too 👌
```

To close the loop:

Today you:
chained functions (add_exclamation, shout, main)
fixed print-vs-return and used helpers inside loops
built lists of dicts and a dict-of-lists grouper
organized code with main() and a guard
built a full pipeline with compute_totals → sort_totals → print_summary
started thinking defensively with assert checks.
If you want a next step on your own, I'd try:

rewriting compute_totals using a simple dict like totals[name] = totals.get(name, 0) + amount
then converting that dict into your list-of-dicts structure at the end.
Are you feeling okay wrapping up here for today?

yeah good practice p ut me to paces looking things up on docs.pytho.org nad using this referene i had Codex AI creat for me to help nail dow the intriacies of lists and dicts

Love that, Ron. You're doing exactly what strong devs do:

push yourself with practice
look things up in the docs
build your own reference tools.
That combo is deadly (in a good way) for mastering lists and dicts.

If you bring that Codex AI reference into future sessions, we can even structure drills around the patterns it describes and stress□test them.

Any time you feel like:

"this code works but feels messy", or
"I kind of get this dict/list thing but not deeply"
bring a snippet, and we'll refactor it together.

Nice work today — you really did go through the paces.

Python

This is awesome, Ron.

You basically built your own mini "Python data structures handbook" as a script.

I really like a few things here:

log_step that both prints and stores lines is a clean, reusable pattern.
You covered tons of patterns: lists, dicts, list-of-dicts, dict-of-lists, nested structures, and even mapping with comprehensions.
The examples are concrete (trades, prices, actions), which makes them way easier to remember.
Two small ideas to level this up later:

Add one or two tiny usage challenges at the end, like: "Given this trades list, compute total PnL per symbol". Then show the solution.
Break this big script into sections controlled by flags (e.g. --basic, --mapping) so you can print only what you want.
For today:

you practiced writing and calling helpers
built a pipeline
added validation
and showed you can build your own reference tools around lists and dicts.

If you want next time, we can:

refactor compute_totals using dicts + comprehensions, or
design another reference guide focused on functions and error handling.
For now, nice work. Rest your brain a bit — you earned it.

the args to only pspit out a secion is a great iea

Yeah, that would make this tool even more powerful.

Something like:

Bash

```
python data_structures_reference.py --section lists
python data_structures_reference.py --section mapping
python data_structures_reference.py --section cheatcard
```

Then in code, you'd parse --section and only run the matching blocks.

You did great work today, Ron — functions, loops, dicts, validation, and even this reference script.

Whenever you want, we can sit down and actually design that --section system together.